Puppet Controlled Hosts Module

RHEL 8 / 9 / 10 - custom OpenVox/Puppet module documentation

Purpose: Maintain the workstation, OpenVox server, and additional controlled records in /etc/hosts.

Module layout

```
custom/
  manifests/hosts.pp
  types/hostentry.pp
  files/ces-puppet-hosts.rb
```

Configuration

```
include custom::hosts

custom::hosts::site_defaults:
  openvox_entry:
    ip: 192.0.2.10
    fqdn: openvox.example.test
    hostname: openvox
    alias: puppet
  additional_entries: []
```

Behavior and safety: Site/network defaults and explicit project overrides are merged before Puppet renders a protected records file. A Ruby helper updates only the Puppet-controlled block in /etc/hosts and provides an idempotent check mode.

Supported platform: Red Hat Enterprise Linux 8, 9, and 10. Generated from the repository sources; rerun **tools/generate_custom_module_docs.ps1** after code changes.

Controlled Hosts

custom/manifests/hosts.pp

```
# @summary
#   Maintains the workstation and controlled host records in /etc/hosts.
#
# @param site_defaults
#   The OpenVox server and optional additional host records for this site and
#   network.
# @param project_overrides
#   Explicit project overrides. Omitted keys retain their site/network default.
#   An empty additional_entries array removes the site/network additional list.
# @param workstation_entry
#   The workstation entry. Its IP address, FQDN, and hostname default to facts.
class custom::hosts (
  Optional[Struct[{
    'openvox_entry'                => Custom::HostEntry,
    Optional['additional_entries'] => Array[Custom::HostEntry],
  }]] $site_defaults = undef,
  Struct[{
    Optional['openvox_entry']      => Custom::HostEntry,
    Optional['additional_entries'] => Array[Custom::HostEntry],
  }] $project_overrides = {},
  Custom::HostEntry $workstation_entry = {
    'ip'       => $facts['networking']['ip'],
    'fqdn'     => $facts['networking']['fqdn'],
    'hostname' => $facts['networking']['hostname'],
  },
) {
  if $site_defaults == undef {
    fail('custom::hosts::site_defaults must be set in site/network Hiera data')
  }

  $effective_settings = $site_defaults + $project_overrides
  $additional_entries = $effective_settings['additional_entries'] ? {
    undef   => [],
    default => $effective_settings['additional_entries'],
  }
  $host_entries = [
    $workstation_entry,
    $effective_settings['openvox_entry'],
  ] + $additional_entries
  $records = $host_entries.map |Custom::HostEntry $entry| {
    $host_alias = $entry['alias'] ? {
      undef   => '',
      default => $entry['alias'],
    }
    [$entry['ip'], $entry['fqdn'], $entry['hostname'], $host_alias].join("\t")
  }
  $records_text = $records.join("\n")
  $records_content = "${records_text}\n"

  file { 'CES Puppet hosts configuration directory':
    ensure => directory,
    path   => '/etc/ces-puppet',
    owner  => 'root',
    group  => 'root',
    mode   => '0755',
  }

  file { 'CES Puppet hosts records':
    ensure  => file,
    path    => '/etc/ces-puppet/hosts.records',
    owner   => 'root',
    group   => 'root',
    mode    => '0600',
```

```
    content => $records_content,
    require => File['CES Puppet hosts configuration directory'],
  }

  file { 'CES Puppet hosts helper':
    ensure => file,
    path   => '/usr/local/sbin/ces-puppet-hosts',
    owner  => 'root',
    group  => 'root',
    mode   => '0755',
    source => 'puppet:///modules/custom/ces-puppet-hosts.rb',
  }

  exec { 'apply CES Puppet hosts records':
    command   => '/usr/local/sbin/ces-puppet-hosts --apply /etc/hosts /etc/ces-puppet/hosts.records',
    unless    => '/usr/local/sbin/ces-puppet-hosts --check /etc/hosts /etc/ces-puppet/hosts.records',
    path      => ['/usr/bin', '/bin'],
    logoutput => 'on_failure',
    require   => [
      File['CES Puppet hosts records'],
      File['CES Puppet hosts helper'],
    ],
  }
}
```

```
# @summary
#   A managed /etc/hosts entry with an optional alias.
type Custom::HostEntry = Struct[{
  'ip'              => Pattern[/\A[^\s#]+\z/],
  'fqdn'            => Pattern[/\A[^\s#]+\z/],
  'hostname'        => Pattern[/\A[^\s#]+\z/],
  Optional['alias'] => Pattern[/\A[^\s#]+\z/],
}]
```

```
#!/opt/puppetlabs/puppet/bin/ruby
# frozen_string_literal: true

BANNER = [
  '# Managed by CES Puppet.',
  '# Manual changes will be replaced.'
].freeze

def read_records(path)
  records = File.readlines(path, chomp: true).reject(&:empty?).map do |line|
    fields = line.split("\t", -1)
    valid_fields = fields.length == 4 && fields.take(3).all? { |field| !field.empty? }
    abort("Invalid record in #{path}: expected IP, FQDN, hostname, and optional alias") unless valid_fields

    fields.reject(&:empty?)
  end

  abort("Invalid record file #{path}: expected at least two records") unless records.length >= 2

  hostnames = records.map { |record| record[2].downcase }
  unless hostnames.uniq.length == hostnames.length
    abort("Invalid record file #{path}: managed hostnames must be unique")
  end

  records
end

def managed_content(current, records)
  managed_hostnames = records.map { |record| record[2].downcase }
  lines = current.lines(chomp: true)

  # Remove a previous copy of the managed banner wherever it occurs.
  lines.reject! { |line| BANNER.include?(line) }

  # Replace a hosts entry when one of its names matches a managed hostname.
  # Comments, blank lines, and every unrelated hosts entry remain untouched.
  lines.reject! do |line|
    entry = line.sub(/\s+#.*\z/, '').strip
    next false if entry.empty? || entry.start_with?('#')

    old_names = entry.split(/\s+/).drop(1).map(&:downcase)
    (old_names & managed_hostnames).any?
  end

  body = lines.join("\n")
  body += "\n" unless body.empty? || body.end_with?("\n")

  managed = records.map { |record| record.join("\t") }
  (BANNER + managed).join("\n") + "\n" + body
end

unless ARGV.length == 3 && %w[--check --apply].include?(ARGV[0])
  abort("Usage: #{File.basename($PROGRAM_NAME)} (--check|--apply) HOSTS_FILE RECORDS_FILE")
end

mode, hosts_path, records_path = ARGV
records = read_records(records_path)

if mode == '--check'
  File.open(hosts_path, 'rb') do |file|
    file.flock(File::LOCK_SH)
    current = file.read
    exit(current == managed_content(current, records) ? 0 : 1)
```

```
  end
else
  # Lock and write through the existing inode so concurrent changes cannot
  # race this update and ownership, mode, ACLs, and SELinux context are kept.
  File.open(hosts_path, 'r+b') do |file|
    file.flock(File::LOCK_EX)
    current = file.read
    desired = managed_content(current, records)
    next if current == desired

    file.rewind
    file.write(desired)
    file.truncate(file.pos)
    file.flush
    file.fsync
  end
end
```